Slim Wrist Endurance & Durability Test Rig

ENGINEERING DESIGN, INTERCONNECT DIRECTORY & FIRMWARE REFERENCE MANUAL

1. SYSTEM ARCHITECTURE & KINEMATICS

The Slim Wrist Test Rig is an automated bi-directional rotational endurance testing system. It validates cyclic fatigue, joint wear, and mechanical backlash on prosthetic wrist mechanisms using an ARM Cortex-M4 microcontroller, high-torque microstepping drive, and a low-backlash planetary gearbox.

CONTROLLER & FREQUENCY
STM32F411CEU6 Black Pill (100 MHz)

DRIVER & MOTOR
Leadshine DM860H + NEMA 23 Bipolar

GEAR REDUCTION RATIO
10:1 Low-Backlash Planetary Reducer

ROTATIONAL RESOLUTION
16,000 Steps / 360° (44.44 Steps / Deg)

2. BILL OF MATERIALS (BOM)

COMPONENT	MODEL / SPECS	QTY	APPLICATION / NOTES
Microcontroller	STM32F411CEU6 "Black Pill"	1	Master controller executing motion profiles & U8g2 graphical UI.
Display Module	GoldenMorning 1.3" OLED (SH1106)	1	I2C monochrome display (128×64), 3.3V native logic on PB8/PB9.
Digital Driver	Leadshine DM860H	1	Opto-isolated microstepping driver configured for 1600 pulses/rev.
Stepper Motor	NEMA 23 High-Torque	1	Bipolar 2-phase stepper motor coupled to planetary stage.
Gearbox	10:1 Planetary Reducer	1	Increases wrist output torque and ensures step stability.
Control Buttons	Tactile Switches (12×12 mm)	7	Sequential active-LOW inputs: UP, LEFT, DOWN, RIGHT, OK, BACK, EMG.
Power Supply	Mean Well 24V / 28V DC	1	High dynamic voltage rail supplying DM860H motor power.

3. ZERO-OVERLAP SEQUENTIAL INTERCONNECT DIRECTORY

To prevent wire crossings on 2-layer perfboard, inputs are placed in sequential order down the left header. Each switch connects between its assigned GPIO and a continuous common Ground bus running along the bottom solder layer.

HEADER PIN	STM32 PIN	SIGNAL / FUNCTION	ELECTRICAL STATE	OPERATIONAL ROLE
Left Pin 1	GND	Common Ground Rail	0V Reference	Common bus for all 7 switches & OLED
Left Pin 2	PA0	Switch: UP	`INPUT_PULLUP`	Navigate menu cursor upwards
Left Pin 3	PA1	Switch: LEFT	`INPUT_PULLUP`	Value decrement (-) / setting toggle
Left Pin 4	PA2	Switch: DOWN	`INPUT_PULLUP`	Navigate menu cursor downwards

HEADER PIN	STM32 PIN	SIGNAL / FUNCTION	ELECTRICAL STATE	OPERATIONAL ROLE
Left Pin 5	PA3	Switch: RIGHT	`INPUT_PULLUP`	Value increment (+) / setting toggle
Left Pin 6	PA4	Switch: OK	`INPUT_PULLUP`	Start endurance test / resume run
Left Pin 7	PA5	Switch: BACK	`INPUT_PULLUP`	Emergency stop / test pause / reset
Left Pin 8	PA6	Switch: EMG / AUX	`INPUT_PULLUP`	Manual EMG replica burst trigger
Right Pin 16	PA8	DM860H PUL+	Push-Pull Output	Optocoupler step pulse generation (10 µs)
Right Pin 15	PA9	DM860H DIR+	Push-Pull Output	Rotational direction level control
Right Pin 5	PB8	OLED SCL	I2C1 Hardware Clock	Clock line for SH1106 display driver
Right Pin 4	PB9	OLED SDA	I2C1 Hardware Data	Data line for SH1106 display driver
Right Pin 1	3V3	OLED VCC	3.3V Power Out	Dedicated logic power for OLED

4. DM860H DIP SWITCH CONFIGURATION

> MICROSTEP RESOLUTION (SW5–SW8)
> **1600 Pulses / Rev (SW5:ON, SW6:ON, SW7:OFF, SW8:ON)**

> STANDSTILL CURRENT MODE (SW4)
> **Half Current Enabled (SW4: OFF) to minimize idle heat**

5. RIG OPERATION & MENU NAVIGATION

- **Parameter Adjustment:** Press `UP (PA0)` or `DOWN (PA2)` to move the active selection between **MODE**, **DELAY**, **ANGLE**, and **CYC**.
- **Value Modification:** Press `LEFT (PA1)` or `RIGHT (PA3)` to modify values:
  - **MODE:** Toggles between `SLOW` (2000 µs), `NORMAL` (1000 µs), and `FAST` (500 µs).
  - **DELAY:** Fine-tunes the cruising step delay in 50 µs increments (switches label to `CUSTOM`).
  - **ANGLE:** Adjusts swing amplitude from 0° to 400° in 10° increments.
  - **CYCLES:** Adjusts test target from 100 to 10,000 cycles in 100-cycle steps.
- **Starting Cycling:** Press `OK (PA4)` to begin. The display switches to the live dashboard with cycle progress bar and elapsed count.
- **Pausing / Stopping:** Press `BACK (PA5)` to halt motion immediately and return to the parameter menu.
- **Manual Trigger:** Press `EMG (PA6)` at any time to execute the prosthetic replica pulse burst.

6. PRODUCTION SOURCE CODE (ARDUINO / STM32DUINO)

Requires the U8g2 library by Oliver in Arduino IDE. The sketch implements 3-phase motion profiling (Acceleration Ramp → Constant Cruise → Deceleration Ramp) to protect mechanical gear teeth from shock loading.

```
#include <Arduino.h>
#include <Wire.h>
#include <U8g2lib.h>

// --- OLED DISPLAY INITIALIZATION (SH1106 on PB8=SCL, PB9=SDA) ---
U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);

// --- SEQUENTIAL PIN ALLOCATION ---
const int PIN_SW_UP    = PA0; // UP
const int PIN_SW_LEFT  = PA1; // LEFT  (Decrement -)
const int PIN_SW_DOWN  = PA2; // DOWN
const int PIN_SW_RIGHT = PA3; // RIGHT (Increment +)
const int PIN_SW_OK    = PA4; // OK    (Start / Resume)
const int PIN_SW_BACK  = PA5; // BACK  (Pause / Stop)
const int PIN_SW_EMG   = PA6; // EMG   (Replica burst)

const int PIN_PUL      = PA8;  // Step Pulse -> DM860H PUL+
const int PIN_DIR      = PA9;  // Direction  -> DM860H DIR+
const int PIN_LED      = PC13; // Status Indicator LED (Active LOW)

// --- MOTION CONFIGURATION ---
int targetAngle = 360; // 0 to 400 degrees
int speedMode   = 1;   // 0=SLOW (2000us), 1=NORMAL (1000us), 2=FAST (500us), 3=CUSTOM
const char* speedNames[] = {"SLOW", "NORMAL", "FAST", "CUSTOM"};
uint32_t cruiseDelayUs   = 1000;
int targetCycles         = 1000;

const uint32_t RAMP_STEPS     = 1000;
const uint32_t START_DELAY_US = 3500;
const uint32_t DWELL_MS       = 800;

int currentSelection = 0; // 0=MODE, 1=DELAY, 2=ANGLE, 3=CYC
bool isRunning       = false;
uint32_t doneCycles  = 0;

void sendStep(uint32_t delayUs) {
  digitalWrite(PIN_PUL, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_PUL, LOW);
  delayMicroseconds(delayUs);
}

void run3PhaseSwing(bool forward, uint32_t cruiseSteps, uint32_t cruiseDelay) {
  digitalWrite(PIN_DIR, forward ? HIGH : LOW);
  delayMicroseconds(50);

  // Phase 1: Soft Launch
  for (uint32_t s = 0; s < RAMP_STEPS; s++) {
    if (digitalRead(PIN_SW_BACK) == LOW) { isRunning = false; return; }
    uint32_t d = START_DELAY_US - ((START_DELAY_US - cruiseDelay) * s / RAMP_STEPS);
    sendStep(d);
  }
  // Phase 2: Constant Cruise
  for (uint32_t s = 0; s < cruiseSteps; s++) {
    if (digitalRead(PIN_SW_BACK) == LOW) { isRunning = false; return; }
    sendStep(cruiseDelay);
  }
  // Phase 3: Soft Stop
  for (uint32_t s = 0; s < RAMP_STEPS; s++) {
    if (digitalRead(PIN_SW_BACK) == LOW) { isRunning = false; return; }
    uint32_t rem = RAMP_STEPS - s;
    uint32_t d = START_DELAY_US - ((START_DELAY_US - cruiseDelay) * rem / RAMP_STEPS);
    sendStep(d);
  }
}

void drawMenuScreen() {
  u8g2.setFont(u8g2_font_6x12_tr);
  u8g2.drawStr(16, 10, "WRIST RIG SETUP");
```

7. GITHUB REPOSITORY STORAGE GUIDE (ZERO-COMMAND-LINE)

To store, back up, and version this project without installing command-line tools or terminal software:

1. **Create Repository:** Log into github.com, click **+** (top-right) → **New repository**. Name it `slim-wrist-test-rig`, check *"Add a README file"*, and click **Create repository**.
2. **Upload Files Directly:** On the repository page, click **Add file** → **Upload files**.
3. **Select Project Files:** Drag and drop your Arduino `.ino` file and this documentation PDF directly into the browser upload box.
4. **Save Changes:** Click the green **Commit changes** button at the bottom. Your project is now safely backed up in the cloud.
5. **Future Updates:** When you modify your code, simply click **Add file** → **Upload files** again and commit. GitHub will automatically track the version history.